Web Programming (IT23502)

Mini Project Report (Unit IV and Unit V)

PlaceTrack

A Campus Placement Drive Tracker

Spring Boot · Spring MVC · JPA / Hibernate · REST · Microservice

Member	Name	Register number	Module
M1	Selvavignesh S	2024115090	Data layer, inheritance, HQL, Session demo, eligibility microservice, company and drive pages
M2	Prakash B	2024115074	Login and roles, apply flow, eligibility client, @Transactional, AOP, DI demo
M3	Krishna K	2024115100	Applicants and status changes, profile, dashboard, REST API, Postman, README, report

Source code: https://github.com/Selvavignesh-S/placetrack2

Department of Information Science and Technology

5th semester

October 2026

Contents

1. Abstract 4
2. Problem Statement 4
3. Objectives 4
4. Scope and Features 5
4.1 Coordinator features 5
4.2 Student features 5
4.3 Kept out of scope 5
5. Technology Stack 6
6. System Architecture 7
6.1 Layers of the main application 7
6.2 Monolith and microservices 8
7. Database Design 9
7.1 Inheritance mapping 9
7.2 ID generation strategy 10
7.3 Sample records 10
8. Implementation 11
8.1 Entities and ORM mapping 11
8.2 Repositories and HQL 11
8.3 SessionFactory, Session and Transaction 12
8.4 Services and business rules 12
8.5 MVC controllers and pages 13
8.6 Login, roles and input safety 13
8.7 Dependency injection, BeanFactory and ApplicationContext 13
8.8 Aspect-oriented programming (AOP) 14
8.9 Transactions 14
8.10 REST API 14
8.11 Eligibility microservice and the service call 15
9. Web Services, REST and Spring Cloud 16
9.1 REST and SOAP 16
9.2 Spring Cloud 16
10. Testing 17
10.1 Automated tests (JUnit 5, Mockito, MockMvc) 17
10.2 API tests (Postman and Newman) 17

10.3 Manual test cases . . . 18
11. DevOps: Git, GitHub and Maven . . . 19
12. How to Run the Application Locally . . . 19
13. Screenshots . . . 20
13.1 Coordinator . . . 20
13.2 Student . . . 23
13.3 Microservice failure handling and AOP audit log . . . 26
13.4 REST API testing . . . 28
13.5 Automated tests . . . 32
13.6 Database (H2 console) . . . 33
14. Individual Contributions . . . 36
15. Conclusion . . . 36
References . . . 36

1. Abstract

PlaceTrack is a web application that manages campus placement drives. A placement drive is one company recruiting for one job role at the college. Placement coordinators add companies and create drives with eligibility criteria: a minimum CGPA, a maximum number of backlogs, the departments that may apply and an application deadline. Students register, keep their academic profile up to date, browse open drives, check whether they are eligible and apply online.

Eligibility is decided by a second, independent Spring Boot application (a microservice) that runs on its own port. The main application calls it over REST before saving an application, and shows the student the exact reasons when a rule fails. If the eligibility service is not reachable, the application is not saved and the student is asked to try later. Coordinators move applicants through Applied, Shortlisted, Selected and Rejected; selecting a student also marks the student as placed in the same database transaction. Every application attempt is logged by an aspect (AOP), and REST endpoints tested with Postman expose drives and applications.

The project is built with Spring Boot 3.5, Spring MVC with Thymeleaf, Spring Data JPA with Hibernate, an H2 database, Bean Validation, Spring AOP, JUnit and Mockito, Maven, Git and GitHub.

2. Problem Statement

In many colleges, placement drives are announced through messages and spreadsheets. Students have to work out by themselves whether they meet each company's criteria, coordinators collect registrations by hand, and it is hard to see who applied where, who was shortlisted and who is already placed. Mistakes are common: ineligible students register, eligible students miss deadlines, and an already placed student may keep applying.

The project builds one simple system in which the criteria are stored with each drive, eligibility is checked automatically when a student applies, and every application has a clear status that both the student and the coordinator can see.

3. Objectives

- Build a working Spring Boot web application using the MVC pattern, created and run with Spring Tool Suite.
- Store data with JPA and Hibernate using annotated entities, an ORM mapping with relationships, an ID generation strategy, CRUD operations and an inheritance mapping.
- Write HQL queries and demonstrate SessionFactory, Session and Transaction.
- Use dependency injection throughout and demonstrate BeanFactory and ApplicationContext.
- Apply AOP to a practical problem: auditing every application attempt.
- Provide REST APIs using GET and POST and test them with Postman.
- Build a second, independent service on another port and call it over REST, including the case where it is down.
- Maintain the code with Git and GitHub and build it with Maven.

4. Scope and Features

4.1 Coordinator features

- Log in and see a dashboard with the number of students, placed students, open drives and applications per drive.
- Add, edit and delete companies. Company names must be unique and a company that has drives cannot be deleted.
- Create, edit and delete drives with their criteria. A drive that already has applications cannot be deleted.
- See the applicants of a drive (best CGPA first) and move each one: Applied to Shortlisted or Rejected, Shortlisted to Selected or Rejected. Selecting a student marks the student as placed.

4.2 Student features

- Register and log in.
- See open drives (deadline today or later) and check eligibility for any drive without applying.
- Apply. The application is refused with a clear message if the student is already placed, the deadline has passed, the student already applied, or an eligibility rule fails.
- See their applications with their status, and withdraw while the status is Applied or Shortlisted.
- Edit their name, department, CGPA and backlogs.

4.3 Kept out of scope

To keep the project small and fully explainable, the following were deliberately left out: Spring Security and JWT (a simple session-based login and an interceptor are used instead), file uploads, e-mail notifications, a JavaScript front end, Docker, and running Spring Cloud components. These are discussed in the report where relevant.

5. Technology Stack

Area	Choice	Purpose
Language	Java 17 (built with JDK 21)	Language level used by both projects
Framework	Spring Boot 3.5.16	Auto-configuration, embedded Tomcat server
Web	Spring MVC, Thymeleaf, Bootstrap 5	Server-rendered HTML pages
Persistence	Spring Data JPA, Hibernate 6	ORM, repositories, HQL
Database	H2 (file mode)	Embedded SQL database, no installation needed
Validation	Bean Validation (Jakarta)	Rules such as @NotBlank and @Min on forms and JSON
AOP	Spring AOP (AspectJ annotations)	Audit log of application attempts
Passwords	BCrypt (spring-security-crypto)	One-way salted password hashes
Service call	RestTemplate with timeouts	Main app calls the eligibility microservice
Testing	JUnit 5, Mockito, MockMvc, Postman, Newman	Unit, web layer and API tests
Build and DevOps	Maven (wrapper), Git, GitHub	Build, dependency management, version control
IDE	Spring Tool Suite 4 / IntelliJ IDEA	Development and running the applications

6. System Architecture

The system consists of two Spring Boot applications that run as separate processes on separate ports. They share no Java code; they only agree on a URL and a JSON format.

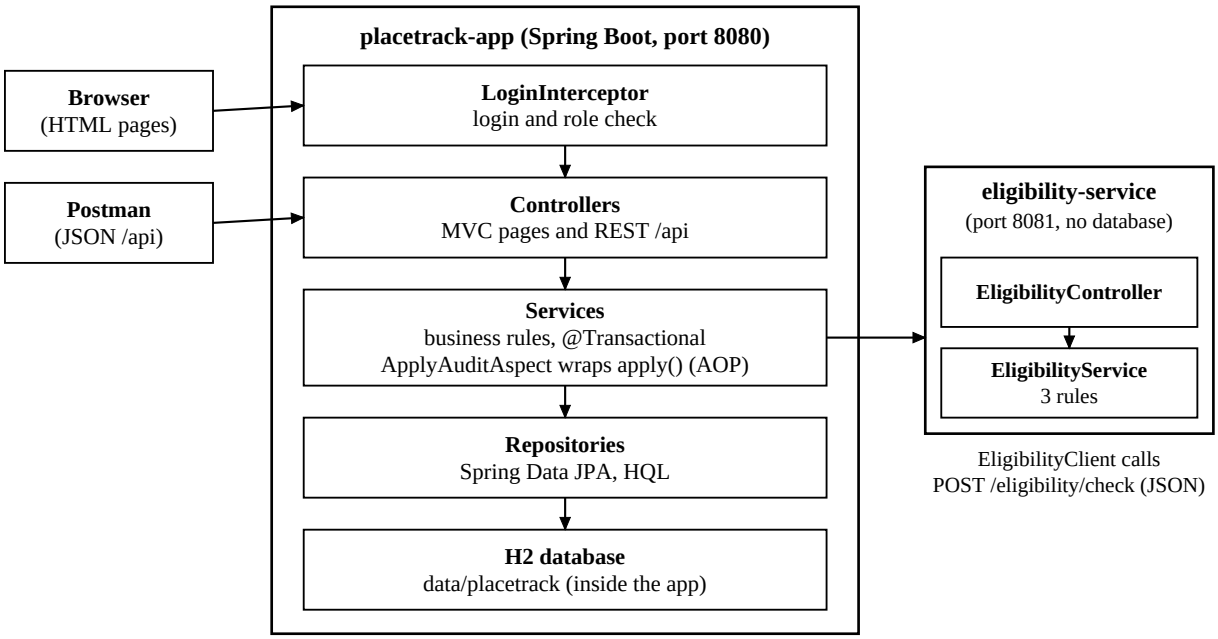


Figure 1: Architecture of PlaceTrack

- **placetrack-app (port 8080)** owns all data and pages. Requests pass through a login interceptor, then a controller, which calls a service, which uses repositories to read and write the H2 database.
- **eligibility-service (port 8081)** is a small stateless service with no database. It receives a student's marks and a drive's criteria and answers whether the student is eligible, with the reasons.
- The main app calls the service through **EligibilityClient** (RestTemplate, 2 second connect and 3 second read timeouts). Postman can also call the service directly, which shows it is independent.

6.1 Layers of the main application

Layer	Package	Responsibility
View	templates/	Thymeleaf HTML pages
Controller	controller/, api/	Handle requests, call services, return a page or JSON
Service	service/	Business rules and transactions
Repository	repository/	Database access with Spring Data JPA and HQL
Entity	model/	Java classes mapped to tables
Supporting	config/, client/, aspect/, exception/	Login check, seed data, REST client, AOP, errors

6.2 Monolith and microservices

A monolith is one application that is built and deployed as a whole. Microservices split the system into small independent services, each with its own process, that talk over the network. Microservices can be changed, scaled or restarted separately, but add network calls, timeouts and failure handling.

PlaceTrack is mostly a monolith with one responsibility, the eligibility decision, extracted into a microservice. This shows the main ideas (separate process and port, REST communication, independent deployment, handling a service that is down) while keeping the project small.

7. Database Design

The database has six tables. Hibernate creates them from the entity classes (*spring.jpa.hibernate.ddl-auto=update*) in an H2 database stored in the *data/* folder.

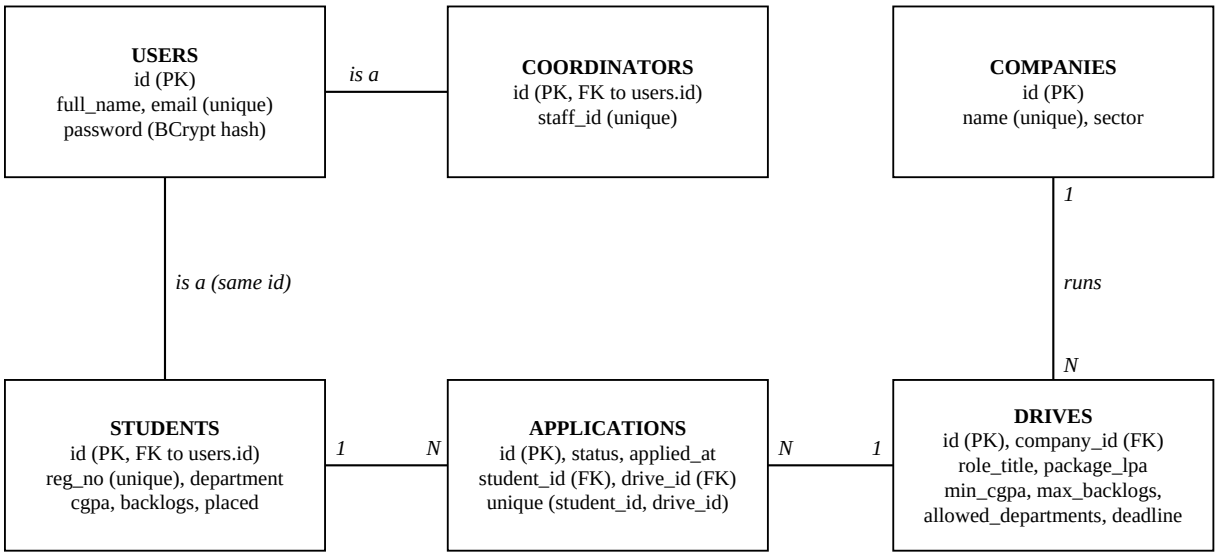


Figure 2: Tables and relationships

Table	Main columns	Notes
users	id, full_name, email, password	Shared part of every account; email unique; password is a BCrypt hash
students	id, reg_no, department, cgpa, backlogs, placed	One row per student; id is also a foreign key to users.id
coordinators	id, staff_id	One row per coordinator; id is also a foreign key to users.id
companies	id, name, sector	name unique
drives	id, company_id, role_title, package_lpa, min_cgpa, max_backlogs, allowed_departments, deadline	company_id is a foreign key; allowed_departments is stored as text such as CSE,IT
applications	id, student_id, drive_id, status, applied_at	Links a student to a drive; unique (student_id, drive_id); status stored as text

7.1 Inheritance mapping

Students and coordinators are both users. The abstract class *User* is mapped with *@Inheritance(strategy = InheritanceType.JOINED)*: the shared fields are stored in *users* and each subclass has its own table containing only its extra fields, linked by the same id. Reading a student therefore joins two tables. JOINED was chosen over SINGLE_TABLE because it avoids a single wide table with many empty columns, and over TABLE_PER_CLASS because shared fields are stored once.

7.2 ID generation strategy

All primary keys use *@GeneratedValue(strategy = GenerationType.IDENTITY)*: the database auto-numbers new rows. In the generated schema this appears as *ID BIGINT GENERATED BY DEFAULT AS IDENTITY*.

7.3 Sample records

On first start the class *DataSeeder* fills an empty database with demo data (every password is Pass@123). The full schema and data are exported in *docs/report/schema.sql* and *docs/report/schema-and-sample-data.sql*.

Email	Name	Dept	CGPA	Backlogs	Placed
student1@placetrack.edu	Aarav Kumar	IT	8.6	0	No
student2@placetrack.edu	Diya Menon	IT	7.2	1	No
student3@placetrack.edu	Karthik Raman	CSE	8.0	0	Yes
student4@placetrack.edu	Sneha Iyer	MECH	7.9	0	No
student5@placetrack.edu	Rohan Das	ECE	9.1	0	No
student6@placetrack.edu	Priya Natarajan	CSE	6.4	2	No

Company	Role	LPA	Min CGPA	Max backlogs	Departments	Deadline
Zephyr Systems	Software Engineer Trainee	4.5	6.0	2	CSE, IT, ECE	today + 10
Nimbus Cloud	Cloud Engineer	9.0	7.5	0	IT, CSE	today + 7
Quanta Analytics	Data Scientist	14.0	8.5	0	CSE, IT, ECE	today + 14
Zephyr Systems	Support Engineer	3.8	6.0	3	all	today - 3 (closed)
Orion Manufacturing	Graduate Engineer Trainee	4.0	6.5	1	MECH, ECE, EEE, CIVIL	today + 9

Two coordinators (coord1 and coord2), five companies (Helio Power has no drives) and four sample applications in different states are also created.

8. Implementation

This chapter explains how each required concept is implemented. Class names refer to the source code in the GitHub repository.

8.1 Entities and ORM mapping

Each entity is a class annotated with *@Entity* and *@Table*. Fields become columns; *@Column(nullable = false, unique = true)* adds database rules; validation annotations such as *@NotBlank*, *@Min* and *@DecimalMax* are checked when forms or JSON are submitted.

Relationships use *@ManyToOne*: many drives belong to one company (column *company_id*), and an application belongs to one student and one drive. The application status is an enum stored as text with *@Enumerated(EnumType.STRING)*, and *@JsonIgnore* keeps password hashes out of JSON.

```
@Entity
@Table(name = "applications",
        uniqueConstraints = @UniqueConstraint(columnNames = {"student_id", "drive_id"}))
public class Application {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @ManyToOne(optional = false) private Student student;   // column student_id
    @ManyToOne(optional = false) private Drive drive;       // column drive_id
    @Enumerated(EnumType.STRING) private ApplicationStatus status;
    private LocalDateTime appliedAt;
}
```

8.2 Repositories and HQL

Repositories are interfaces that extend *JpaRepository*, which provides save, findById, findAll, deleteById and count (CRUD) without any code. Extra queries are either derived from the method name (for example *existsByEmail*) or written in HQL with *@Query*. HQL uses class and field names, and Hibernate translates it to SQL.

Repository method	Kind	Purpose
DriveRepository. findOpenDrives(today)	HQL	Drives whose deadline is today or later
ApplicationRepository. findByStudent(id)	HQL	A student's applications, newest first
ApplicationRepository. findApplicantsOfDrive(id)	HQL, join fetch	Applicants with their student loaded in one query, best CGPA first
ApplicationRepository. countApplicationsPerDrive()	HQL, GROUP BY, left join	Rows of company, role and count for the dashboard; drives with no applications are kept
UserRepository. findByEmail(email)	Derived	Login
ApplicationRepository. existsByStudentIdAndDriveId	Derived	Duplicate application check
CompanyRepository.findByNameIgnoreCase, DriveRepository.countByCompanyId	Derived	Unique company name; company with drives cannot be deleted

```
@Query("select d from Drive d where d.deadline >= :today order by d.deadline")
List<Drive> findOpenDrives(@Param("today") LocalDate today);

@Query("select a from Application a join fetch a.student s "
     + "where a.drive.id = :driveId order by s.cgpa desc")
List<Application> findApplicantsOfDrive(@Param("driveId") Long driveId);
```

8.3 SessionFactory, Session and Transaction

The **SessionFactory** is created once from the entity mappings and database settings; it is heavy, thread-safe and creates Sessions. A **Session** is one unit of work with the database: it wraps a connection and keeps a first-level cache of loaded objects. A **Transaction** groups changes so that they are either all saved (commit) or all undone (rollback). In normal code Spring manages these through repositories and *@Transactional*.

The class *HibernateSessionDemo* (run with the *demo* profile) uses them directly: it obtains the SessionFactory from JPA, opens a Session, begins a Transaction, runs an HQL query, inserts a company, rolls back and shows that the insert was undone. Its console output:

```
===== Hibernate demo: SessionFactory -> Session -> Transaction =====
1. HQL GROUP BY, students per department:
     CSE = 2
     ECE = 1
     IT = 2
     MECH = 1
2. Companies before insert: 5
3. Companies after insert (inside the transaction): 6
4. tx.rollback() called
5. Companies seen by a NEW session: 5  -> the insert was undone
```

8.4 Services and business rules

Services hold the rules; controllers never use repositories directly. The most important method is *ApplicationService.apply*, which runs inside one transaction and checks the rules in this order (cheap database checks first, the network call last):

- The student must not already be placed.
- The drive's deadline must not have passed.
- The student must not have applied to this drive already (also enforced by a unique constraint).
- The eligibility service must accept the student: CGPA at least the minimum, backlogs at most the limit, department in the allowed list.
- If every rule passes, the application is saved with status APPLIED. If any rule fails, an exception is thrown, the transaction rolls back and nothing is saved.

Rule	Where it is enforced
Company names are unique; a company with drives cannot be deleted	CompanyService
A drive with applications cannot be deleted	DriveService
Eligibility: CGPA, backlogs, department	eligibility-service (EligibilityService)
Deadline, one application per drive, placed students cannot apply	ApplicationService.apply
Withdraw only own application, only while Applied or Shortlisted	ApplicationService.withdraw

Rule	Where it is enforced
Allowed status moves; Selected also marks the student as placed	ApplicationService.updateStatus

8.5 MVC controllers and pages

Controllers (*@Controller*) handle a URL, put data into the *Model* and return a template name; Thymeleaf fills the template and returns HTML. Forms use the Post-Redirect-Get pattern: after a successful POST the browser is redirected, and a one-time flash message is shown, so refreshing the page does not submit the form again.

Controller	Main URLs	Used by
AuthController	/, /login, /logout, /register	Everyone
DashboardController	/coordinator/dashboard	Coordinator
CompanyController	/coordinator/companies (list, add, edit, delete)	Coordinator
DriveController	/coordinator/drives (list, create, edit, delete)	Coordinator
ApplicantController	/coordinator/drives/{id}/applicants, /coordinator/applications/{id}/status	Coordinator
StudentController	/student/drives, /student/drives/{id}/check, /apply, /student/applications	Student
ProfileController	/student/profile	Student

8.6 Login, roles and input safety

- Passwords are stored as BCrypt hashes. At login, *AuthService* compares the typed password with the hash and stores a small *SessionUser* (id, name, role) in the HTTP session.
- The session id is changed at login (*request.changeSessionId()*) to prevent session fixation.
- *LoginInterceptor* runs before every /student/** and /coordinator/** page: a visitor who is not logged in is sent to the login page and a user with the wrong role gets 403 Forbidden.
- Forms bound to entities use *@InitBinder* with an allow-list of fields, so extra fields such as *placed=true* or another *id* added to a request are ignored.

8.7 Dependency injection, BeanFactory and ApplicationContext

Every service and controller receives the objects it needs through its constructor; Spring creates them (beans) and passes them in. No class creates its collaborators with *new*, which keeps classes loosely coupled and lets tests pass in mock objects instead.

The class *BeanFactoryDemo* shows the two containers. A **BeanFactory** is the basic container and creates a bean only when it is requested. An **ApplicationContext** extends it, creates singleton beans at startup and adds annotation processing, AOP, events and profiles; Spring Boot creates one for the application. Output of the demo:

```
===== Part 1: BeanFactory (lazy) =====
1. Bean definition registered. Nothing created yet.
2. Calling factory.getBean("greeter")...
      -> Greeter constructor runs (bean created)
3. Hello from a Spring bean

===== Part 2: ApplicationContext (eager + dependency injection) =====
1. Beans registered. Calling context.refresh()...
      -> Greeter constructor runs (bean created)
2. refresh() finished: the Greeter was already created above, before any getBean call.
3. Hello from a Spring bean, injected into WelcomeService
```

8.8 Aspect-oriented programming (AOP)

Auditing every application attempt is a cross-cutting concern: it is not part of the business logic. The aspect *ApplyAuditAspect* uses an *@Around* advice with the pointcut *execution(* com.placetrack.service.ApplicationService.apply(..))*. Spring wraps the service in a proxy, so every call to apply is timed and logged with the student, the drive and the outcome, while the service itself contains no logging code.

```
@Around("execution(* com.placetrack.service.ApplicationService.apply(..))")
public Object audit(ProceedingJoinPoint joinPoint) throws Throwable {
    Object[] args = joinPoint.getArgs();   // apply(studentId, driveId)
    long start = System.currentTimeMillis();
    try {
        Object result = joinPoint.proceed();    // run the real apply()
        log.info("AUDIT apply student={} drive={} outcome=SUCCESS time={}ms", ...);
        return result;
    } catch (RuntimeException e) {
        log.info("AUDIT apply student={} drive={} outcome=BLOCKED reason=\"{}\" ...", ...);
        throw e;
    }
}
```

8.9 Transactions

@Transactional on a service method makes Spring begin a transaction before the method and commit it afterwards, or roll it back if a runtime exception is thrown. When a coordinator selects a student, *updateStatus* changes two rows in two tables, the application's status and the student's placed flag, and both are committed together. Hibernate's dirty checking writes the changes at commit time without an explicit save call.

8.10 REST API

ApiController is a *@RestController*: it returns JSON instead of HTML and reuses the same services, so the same rules, transaction and audit log apply. *ApiErrorHandler* converts exceptions into JSON errors with suitable status codes.

Method and path	Body	Success	Errors
GET /api/drives	–	200, list of open drives	–
GET /api/drives/{id}	–	200, one drive	404
POST /api/companies	{name, sector}	201, the new company	400
POST /api/applications	{studentId, driveId}	201, the new application	400 with reasons, 404, 503 if the service is down

Method and path	Body	Success	Errors
GET /api/students/{id}/applications	–	200, list	–

For simple testing the API has no login and takes the student id from the request body. A real system would identify the student from the session or a token such as a JWT.

8.11 Eligibility microservice and the service call

The eligibility service has one controller and one service. *POST /eligibility/check* receives the student's marks and the drive's criteria as JSON, validates them with *@Valid* (a missing field gives 400 Bad Request) and returns the decision with one reason per failed rule. *GET /eligibility/ping* returns *{"status":"UP"}*.

```
Request:  { "cgpa": 7.2, "backlogs": 1, "department": "IT",
            "minCgpa": 7.5, "maxBacklogs": 0, "allowedDepartments": ["IT", "CSE"] }
Response: { "eligible": false,
            "reasons": [ "CGPA 7.2 is below the required 7.5",
                         "Backlogs 1 exceed the limit of 0" ] }
```

The main application's *EligibilityClient* sends this request with RestTemplate using 2 second connect and 3 second read timeouts. A refused connection, a timeout or an error reply becomes an *EligibilityUnavailableException*; the student sees "Eligibility check unavailable, try later" (the API returns 503) and nothing is saved.

9. Web Services, REST and Spring Cloud

A web service is software that offers functions to other programs over a network using standard protocols and formats, independent of the programming language. Both the eligibility service and the /api endpoints of the main application are web services.

9.1 REST and SOAP

	REST	SOAP
Nature	Architectural style using plain HTTP	Protocol with strict rules
Message format	Usually JSON (also XML or text)	Always XML in an envelope
Contract	None required (often OpenAPI documentation)	WSDL file describes each operation
Operations	HTTP methods on resources (GET /api/drives)	Named operations, usually all sent with POST
Weight	Light and fast, easy to cache	Heavier, more parsing
Typical use	Web and mobile apps, microservices	Banking, government, legacy enterprise systems

PlaceTrack uses REST: resources at URLs, GET to read and POST to create, standard status codes (200, 201, 400, 404, 503) and JSON bodies. GET is idempotent (repeating it changes nothing); POST is not.

9.2 Spring Cloud

Spring Cloud is a set of projects that solve common problems of systems made of many services. None is used in this project, but each one corresponds to something done by hand here:

Feature	What it does	Relation to PlaceTrack
OpenFeign	Declarative REST client: an annotated interface instead of hand-written HTTP calls	Would replace EligibilityClient
Resilience4j	Circuit breaker: stops calling a failing service for a while and uses a fallback	Would replace the manual timeout and "try later" handling
Eureka	Service discovery: services register by name	Would replace the fixed eligibility.url
Spring Cloud Gateway	One entry point that routes requests to services	Single address for both apps
Config Server	Central configuration for all services	Would replace the separate property files

10. Testing

The project was tested at three levels: automated unit and web-layer tests run by Maven, API tests with Postman and Newman, and manual tests of the web application. Screenshots of the results are in chapter 13.

10.1 Automated tests (JUnit 5, Mockito, MockMvc)

Test class	Type	Tests	What is checked
EligibilityServiceTest	Plain JUnit	5	The three rules, including the equal-value boundary
EligibilityControllerTest	@WebMvcTest	2	200 for valid JSON, 400 for a missing field
RepositoryTest	@DataJpaTest	4	The HQL queries on a temporary database
PlacetrackAppTests	@SpringBootTest	2	The application starts; seed data; inheritance
ApplicationServiceTest	Mockito	8	Apply and withdraw rules; nothing saved when a rule fails
StatusUpdateTest	Mockito	4	Allowed status moves; Selected marks the student as placed
ApiControllerTest	@WebMvcTest, @MockitoBean	6	API status codes and JSON, no password hash in output
Total		31	All passing (mvn clean verify: BUILD SUCCESS)

10.2 API tests (Postman and Newman)

A Postman collection with an environment (*baseUrl*, *eligUrl*) is stored in the repository. Every request has test scripts that check the status code and important fields. The collection was run in the Postman application and from the command line with Newman, Postman's collection runner.

Folder	Requests	Examples
00 Eligibility service	5	Ping; eligible; low CGPA and backlogs; wrong department; missing field gives 400
01 Drives and companies	5	List open drives; one drive; missing drive gives 404; create company 201; no name 400
02 Applications	7	Apply 201; apply again 400; ineligible 400 with reasons; placed 400; expired 400; missing field 400; list applications 200
03 Service down	1	Apply with the eligibility service stopped gives 503

Result: folders 00 to 02 ran 17 requests with 36 assertions and 0 failures; folder 03 returned 503 with both assertions passing.

10.3 Manual test cases

No.	Test	Expected result	Result
T1	Open a student page without logging in	Redirected to the login page	Pass
T2	Log in with a wrong password	"Wrong email or password"	Pass
T3	Coordinator logs in	Dashboard: 6 students, 1 placed, 4 open drives	Pass
T4	Delete a company that has drives	Refused with a message	Pass
T5	Delete a drive that has applications	Refused with a message	Pass
T6	Create a new drive	"Drive saved", drive listed as Open	Pass
T7	Select a shortlisted applicant	Status SELECTED; student marked placed (dashboard Placed 2)	Pass
T8	student2 checks eligibility for Nimbus Cloud	Two reasons: CGPA and backlogs	Pass
T9	student2 applies to Nimbus Cloud	Refused with the reasons, nothing saved	Pass
T10	student1 applies to Nimbus Cloud	"Applied successfully", status APPLIED	Pass
T11	student1 applies to the same drive again	"You have already applied"	Pass
T12	student3 (placed) applies	"You are already placed"	Pass
T13	Apply to the closed drive by URL	"The deadline for this drive has passed"	Pass
T14	Student opens a coordinator page	403 Forbidden	Pass
T15	Apply while the eligibility service is stopped	"Eligibility check unavailable, try later"	Pass
T16	Apply again after restarting the service	Saved as APPLIED	Pass
T17	Withdraw an application	Status WITHDRAWN	Pass
T18	Register with an existing e-mail / a 3-character password	Refused with a message	Pass
T19	Profile: CGPA 11	Validation error, nothing changed	Pass

11. DevOps: Git, GitHub and Maven

DevOps is a way of working in which development and operations cooperate and automate building, testing and releasing software, so that changes reach users quickly and reliably. Its main practices are version control, build automation, automated testing, continuous integration (every change is built and tested automatically) and continuous delivery (tested changes are released automatically).

Tool	Purpose	Used in this project
Git	Version control: history of every change, branches	Yes
GitHub	Shared repository, collaborators, pull requests	Yes (https://github.com/Selvavignesh-S/placetrack2)
Maven with wrapper	Build, dependencies, tests, packaging	Yes (./mvnw clean verify)
JUnit, Mockito, Postman, Newman	Automated testing	Yes
GitHub Actions / Jenkins	Continuous integration pipelines	Not used (Newman and Maven can run there)
Docker	Package an application with its runtime	Not used

The code is organised in one Git repository with two Maven projects. Work was done on separate branches for each member and merged into the main branch. Each project contains the Maven wrapper (mvnw), so Maven does not need to be installed. The Maven lifecycle used is *clean → compile → test → package → verify*: *./mvnw clean verify* compiles the code, runs all tests and builds an executable jar.

12. How to Run the Application Locally

Requirements: JDK 17 or newer and Git. Maven and the database do not need to be installed. Full instructions are in README.md in the repository.

```
git clone https://github.com/Selvavignesh-S/placetrack2.git
cd placetrack2

# terminal 1: eligibility service (port 8081)
cd eligibility-service && ./mvnw spring-boot:run

# terminal 2: main application (port 8080)
cd placetrack-app && ./mvnw spring-boot:run

# open http://localhost:8080   (demo login: coord1@placetrack.edu / Pass@123)
# H2 console: http://localhost:8080/h2-console
#             JDBC URL jdbc:h2:file:./data/placetrack, user sa, empty password
# tests:      ./mvnw test   (in each project folder)
```

On Windows, use *mvnw.cmd* instead of *./mvnw*. In Spring Tool Suite, import both folders with File → Import → Existing Maven Projects and run each with Run As → Spring Boot App.

13. Screenshots

The screenshots below were taken while running both applications locally on fresh demo data.

13.1 Coordinator

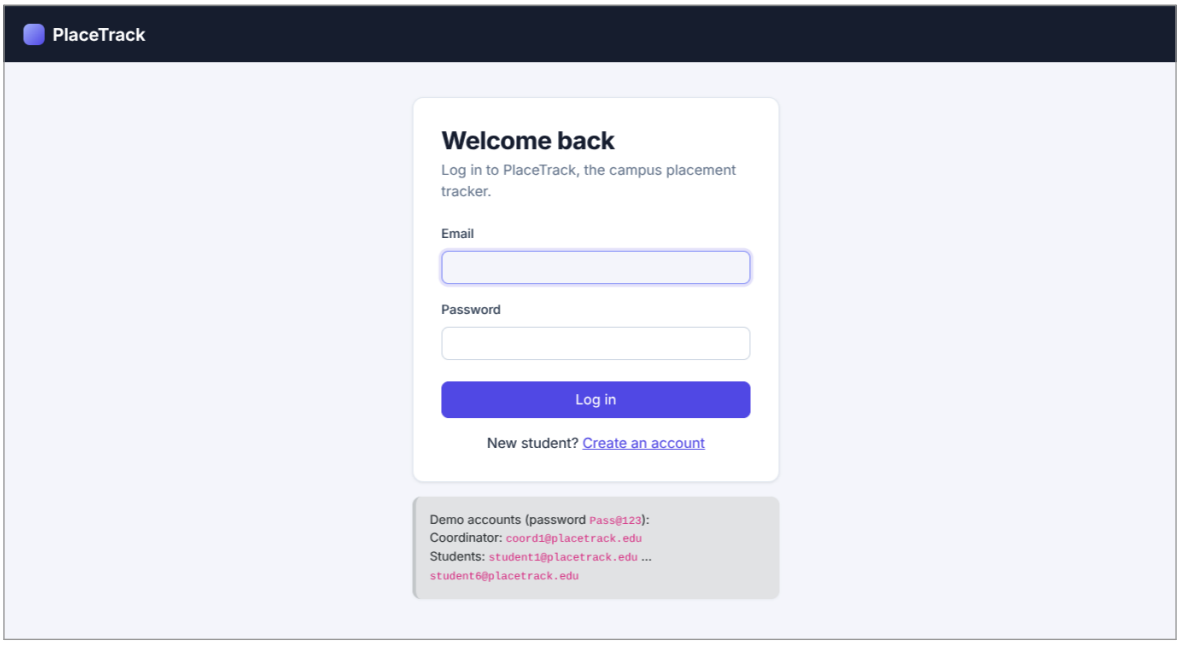


Figure 3: Login page with the demo accounts

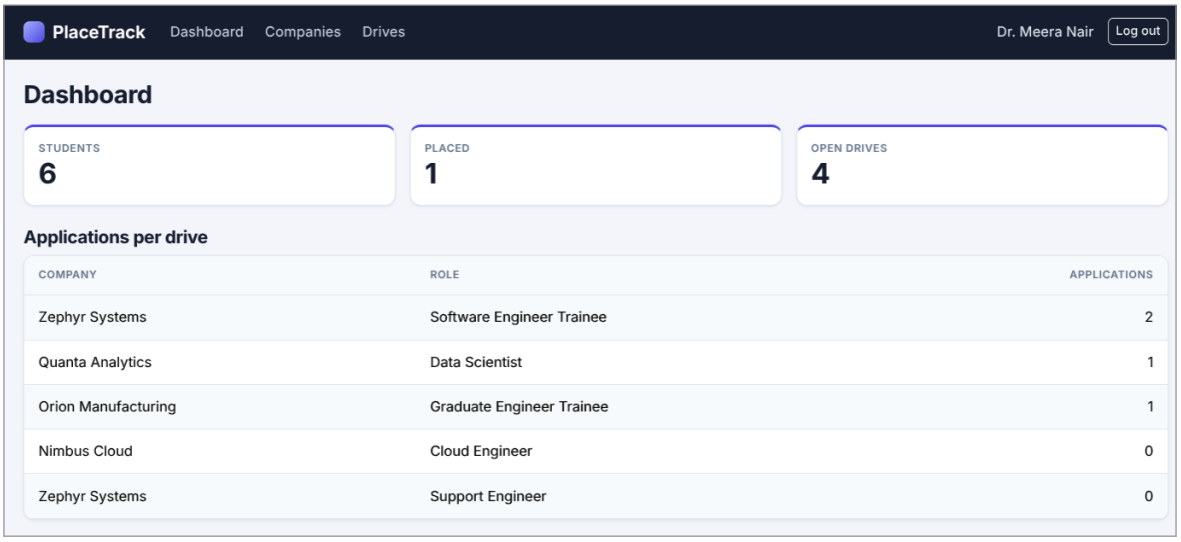


Figure 4: Coordinator dashboard: students, placed students, open drives and applications per drive (HQL GROUP BY query)

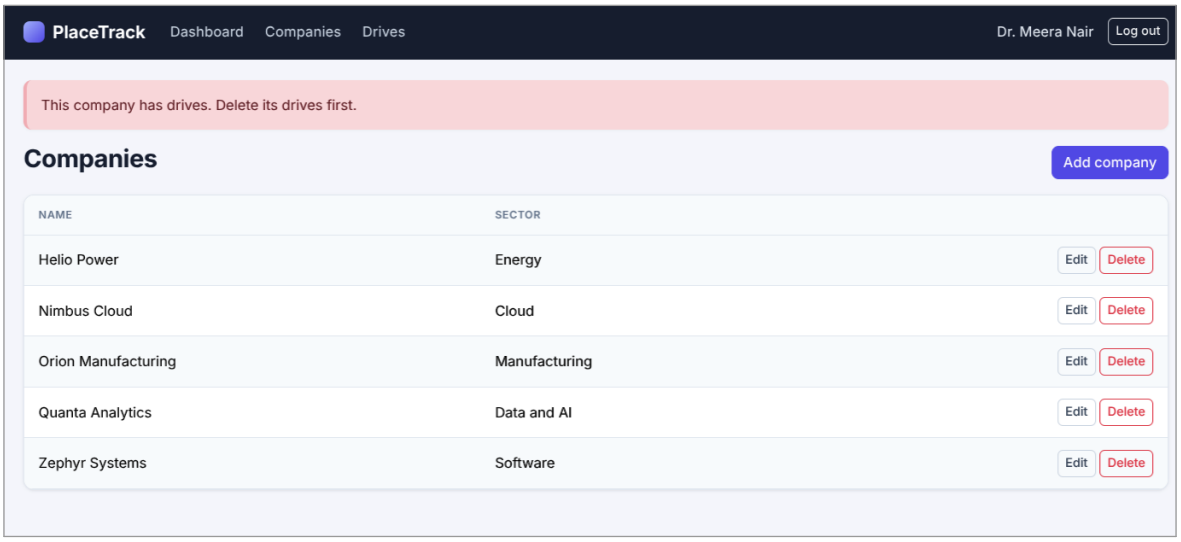


Figure 5: Deleting a company that has drives is refused

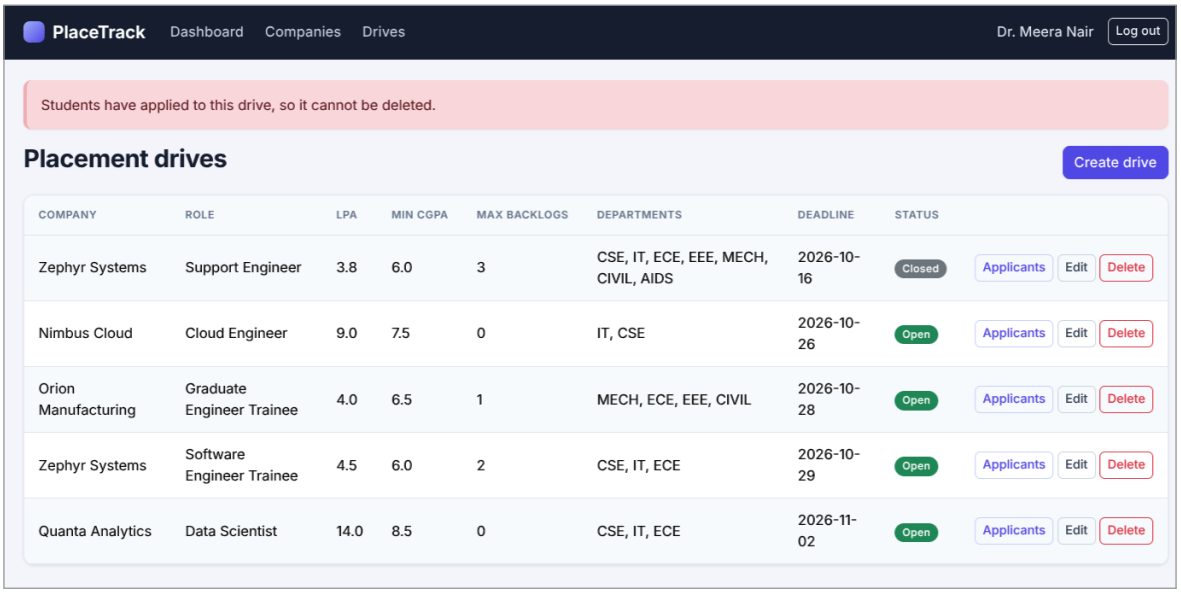


Figure 6: Deleting a drive that already has applications is refused

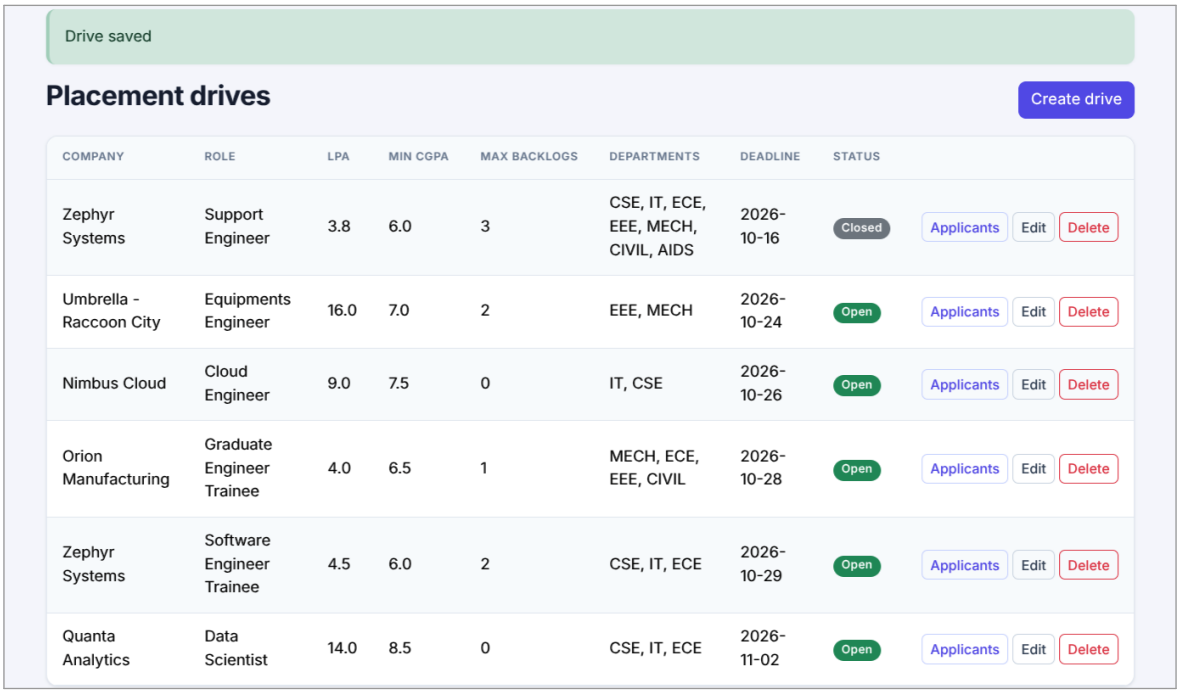


Figure 7: A new drive created by the coordinator appears in the list as Open

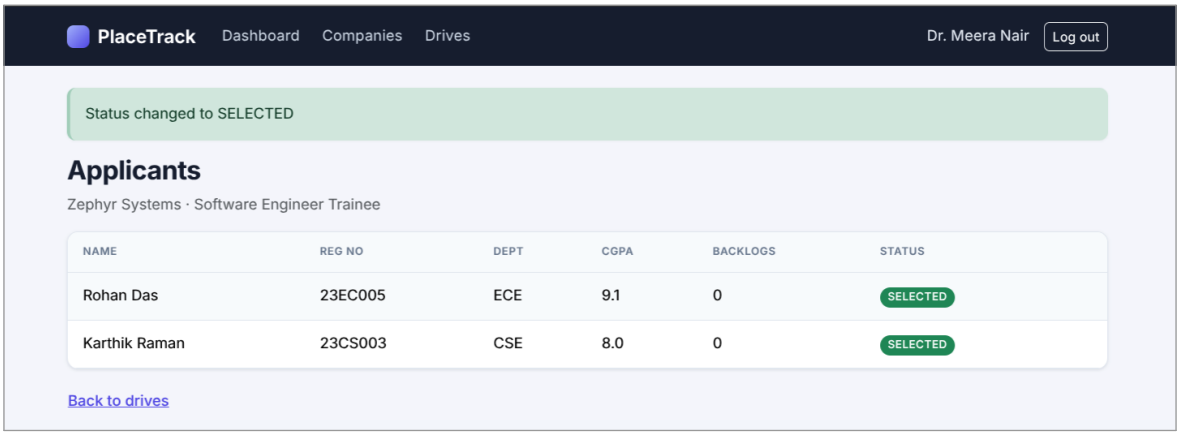


Figure 8: Applicants of a drive after selecting Rohan Das

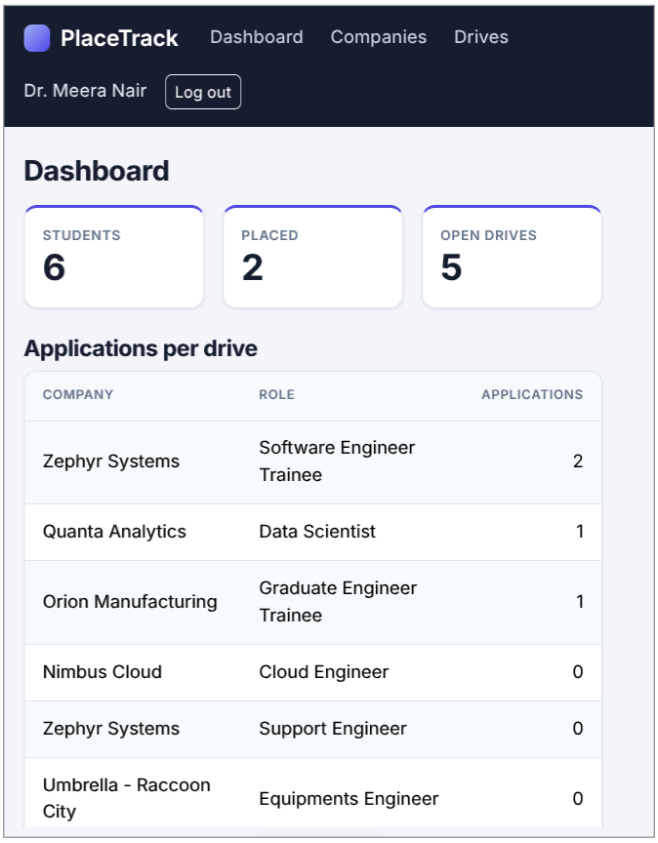


Figure 9: Dashboard after the selection: the number of placed students increased to 2

13.2 Student

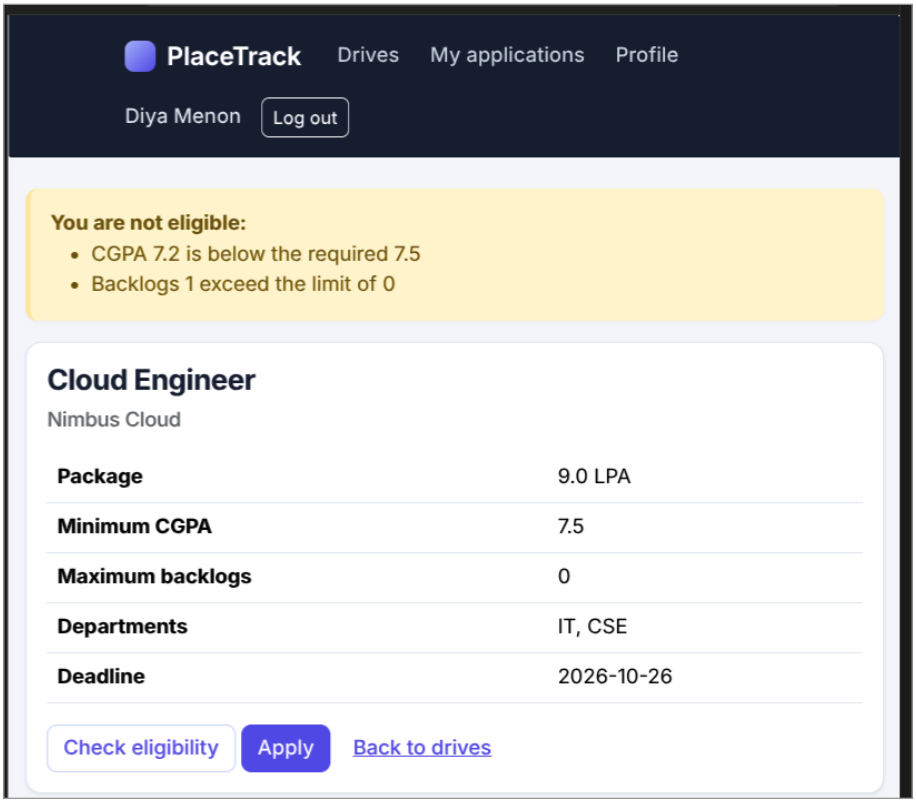


Figure 10: Eligibility check for a student who does not meet the criteria: one reason per failed rule

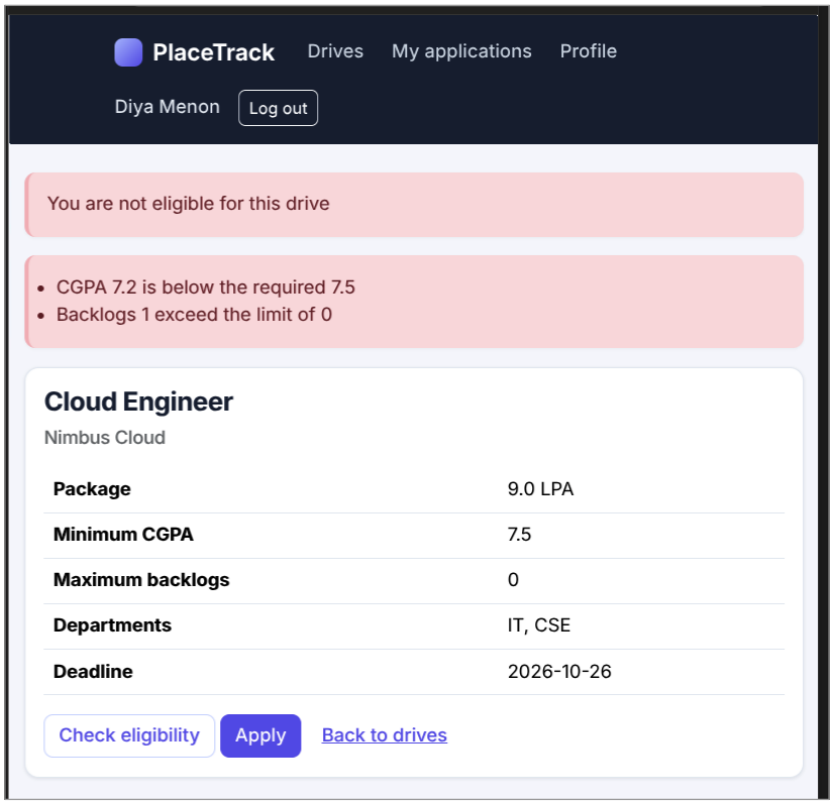


Figure 11: Applying anyway is refused with the same reasons; nothing is saved

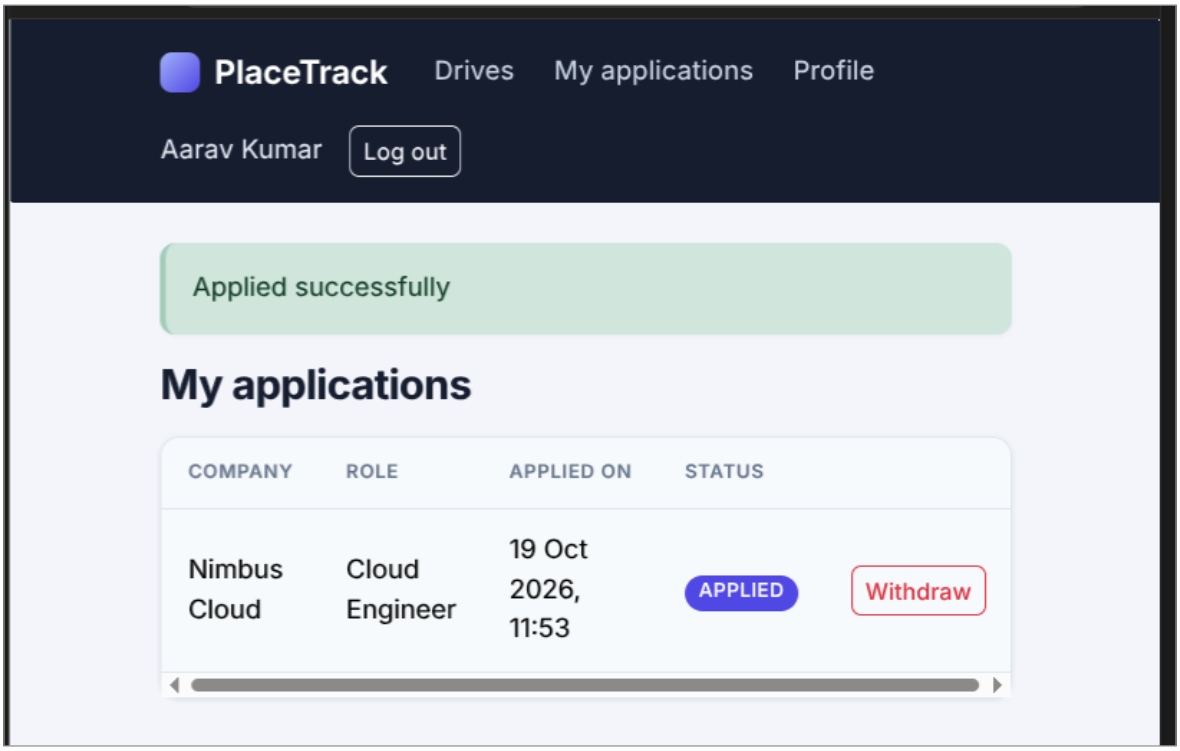


Figure 12: A successful application, shown in My applications with status APPLIED

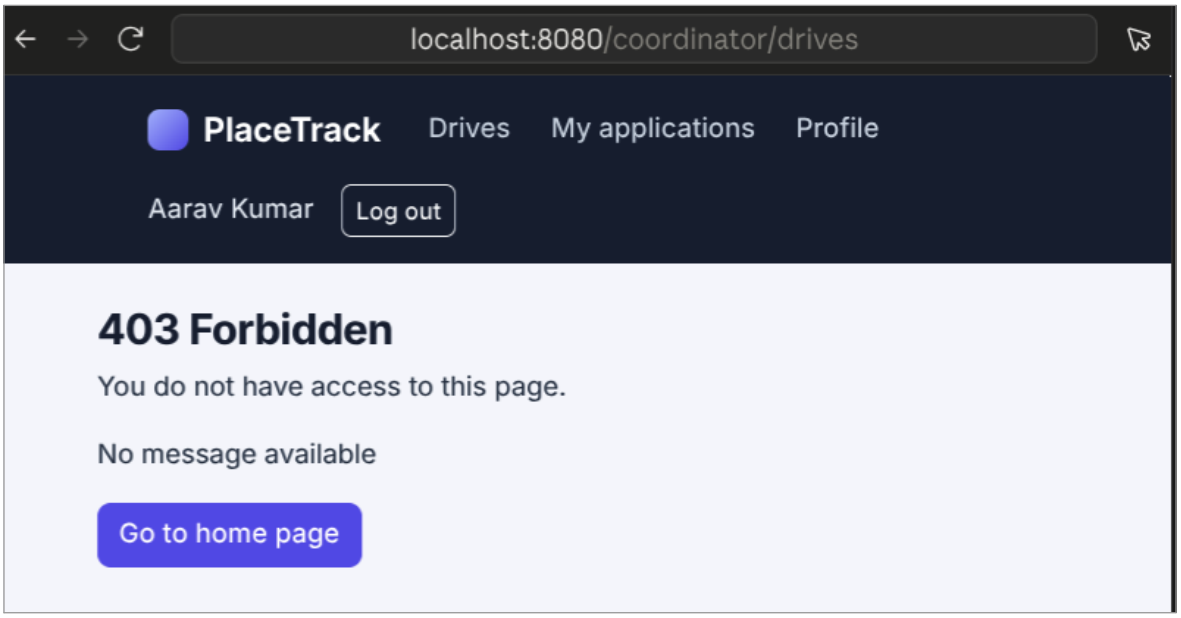


Figure 13: A student opening a coordinator page gets 403 Forbidden

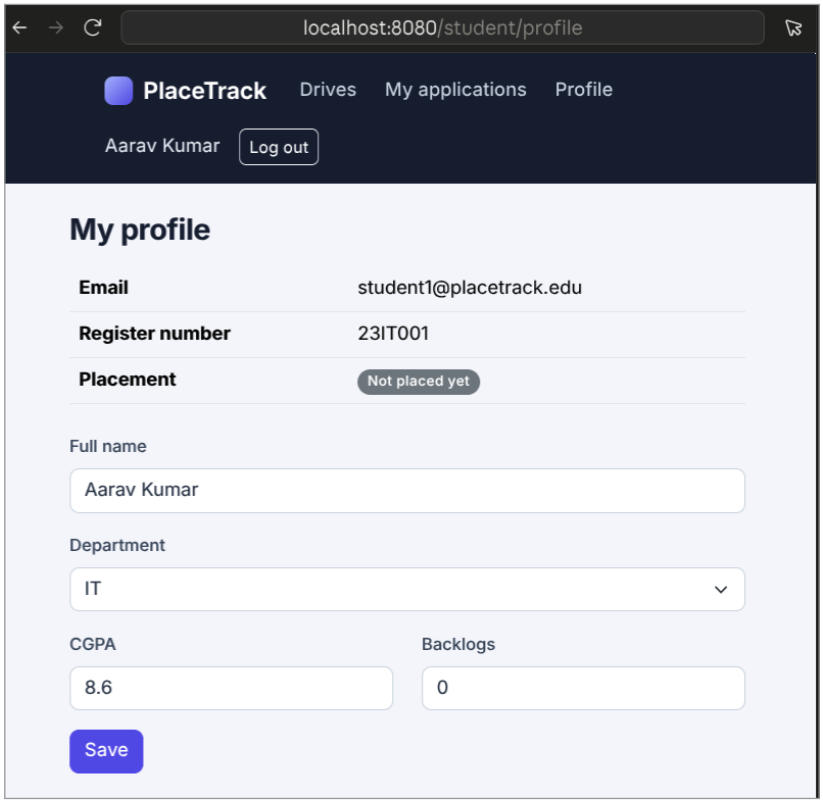


Figure 14: Student profile: e-mail, register number and placement status cannot be edited here

13.3 Microservice failure handling and AOP audit log

```
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  25:23 min
[INFO] Finished at: 2026-10-19T12:01:19+05:30
[INFO] ------------------------------------------------------------------------

~/Documents/Java Mini project./eligibility-service ui-polish* 25m 25s
❯
```

Figure 15: The eligibility service is stopped (Ctrl + C)

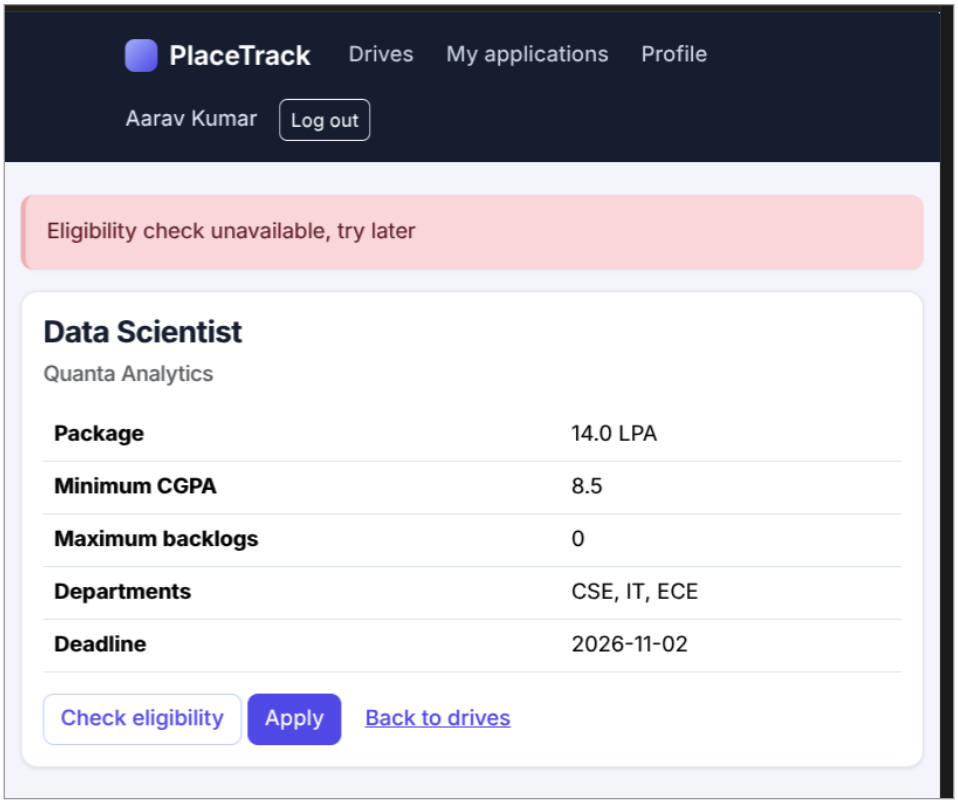


Figure 16: Applying while the service is down: the student is asked to try later and nothing is saved

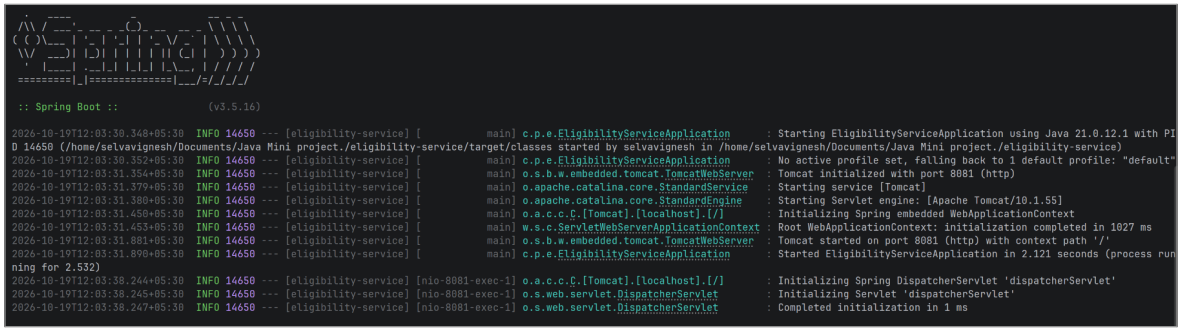


Figure 17: The eligibility service started again on port 8081

```
Tomcat initialized with port 8081 (http)
Tomcat started on port 8081 (http) with context path '/'
Started EligibilityServiceApplication in 2.121 seconds
```

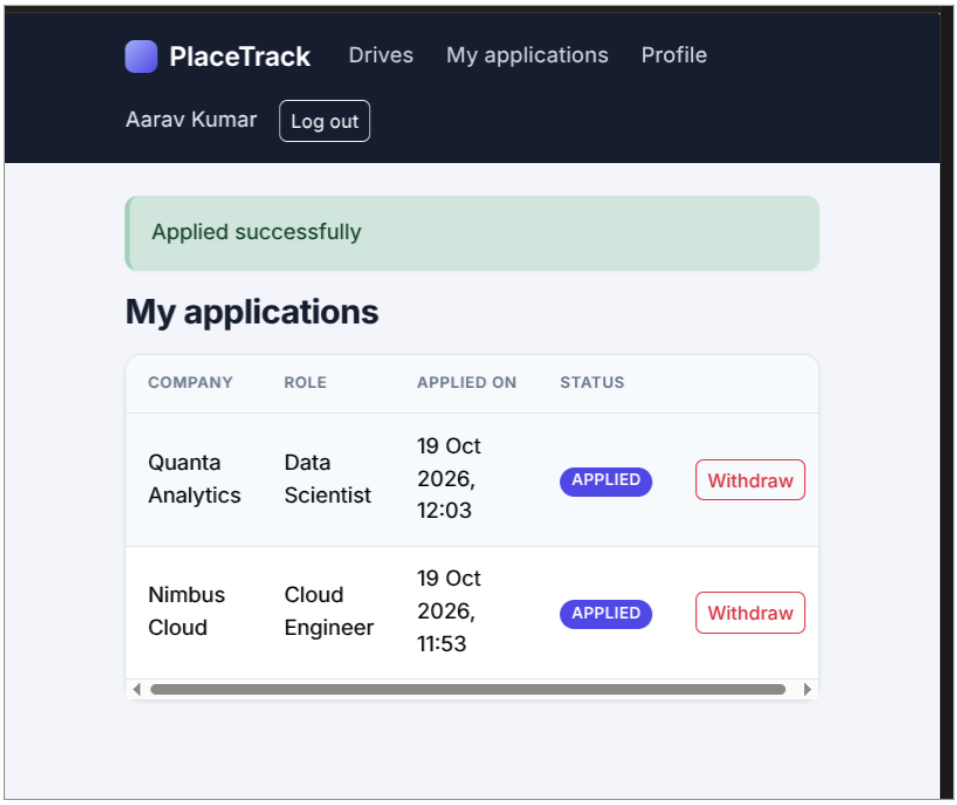


Figure 18: After the restart the same application succeeds

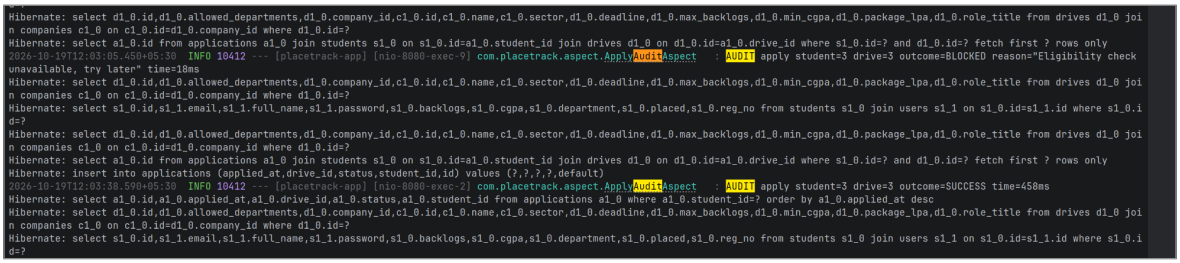


Figure 19: Audit lines written by ApplyAuditAspect: the attempt blocked while the service was down and the successful attempt after the restart (the INSERT appears only for the successful one)

The two AUDIT lines from the screenshot above, as text:

```
12:03:05 INFO [nio-8080-exec-9] com.placetrack.aspect.ApplyAuditAspect :
    AUDIT apply student=3 drive=3 outcome=BLOCKED
          reason="Eligibility check unavailable, try later" time=18ms
Hibernate: insert into applications (applied_at,drive_id,status,student_id,id) ...
12:03:38 INFO [nio-8080-exec-2] com.placetrack.aspect.ApplyAuditAspect :
    AUDIT apply student=3 drive=3 outcome=SUCCESS time=458ms
```

13.4 REST API testing

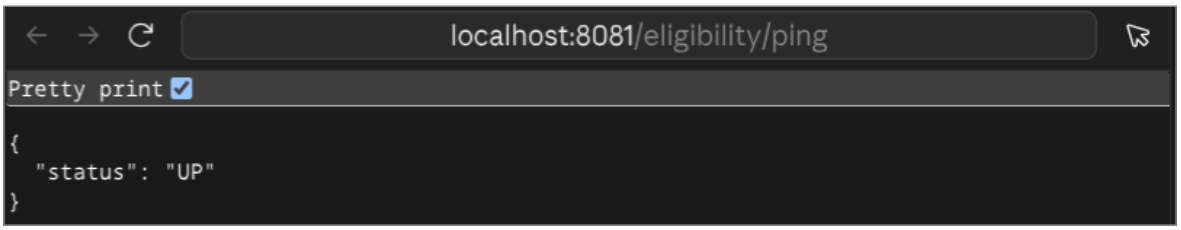


Figure 20: GET /eligibility/ping in the browser

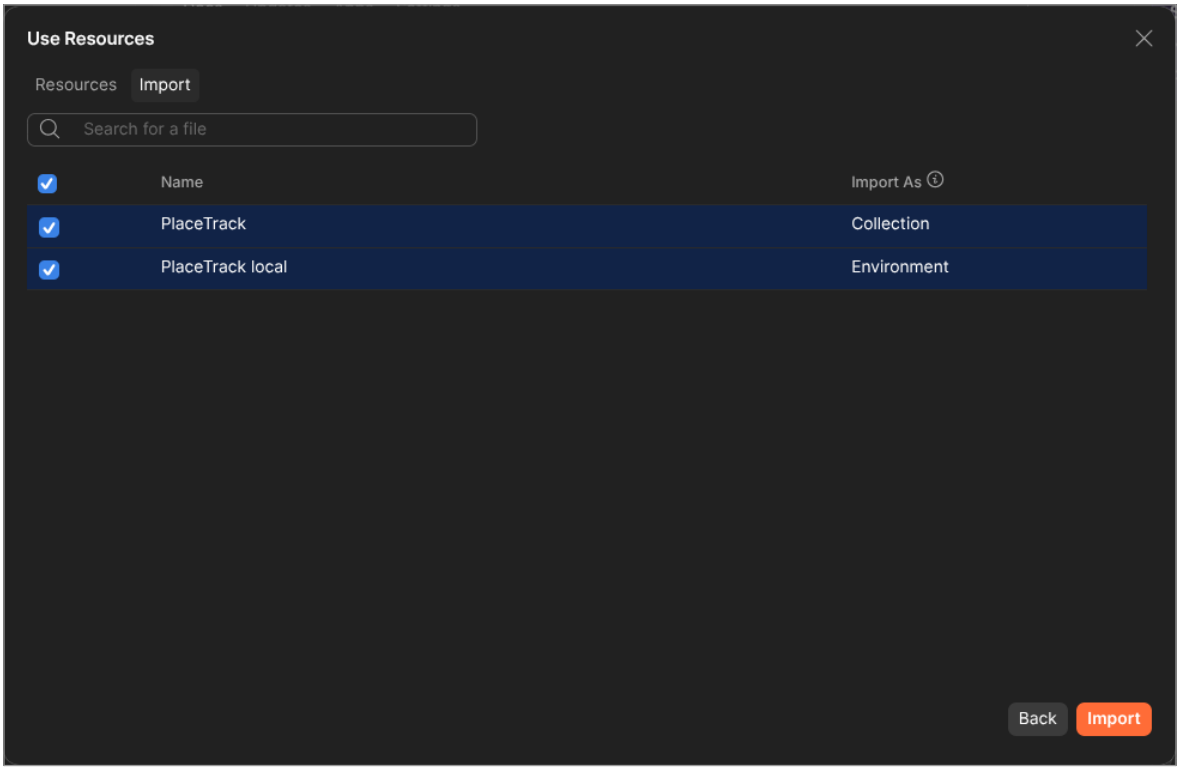


Figure 21: Importing the PlaceTrack collection and environment into Postman

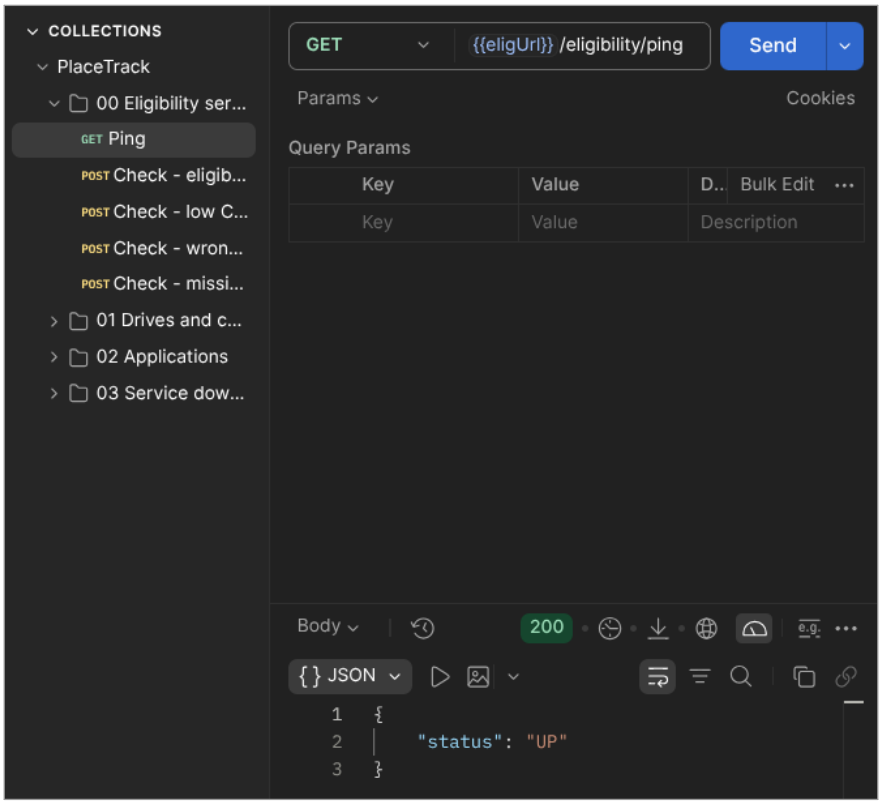


Figure 22: Postman: GET request to the eligibility service (200 OK)

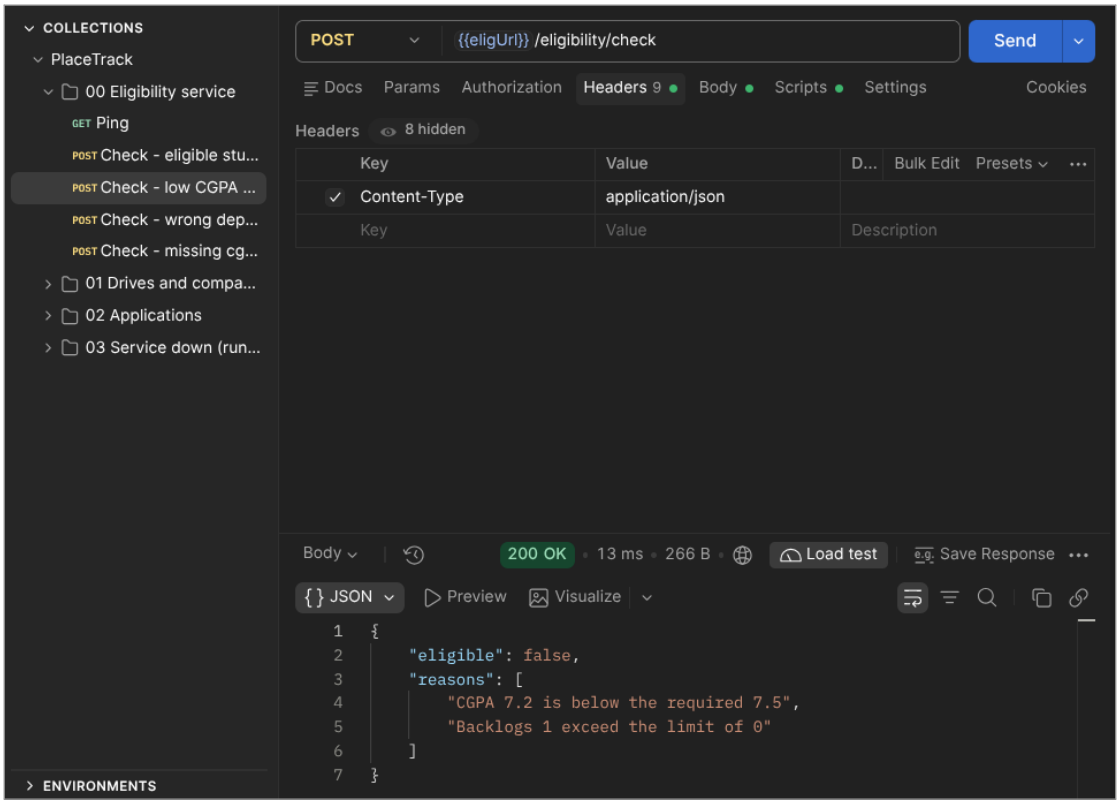


Figure 23: Postman: POST /eligibility/check returns the decision with the reasons (200 OK)

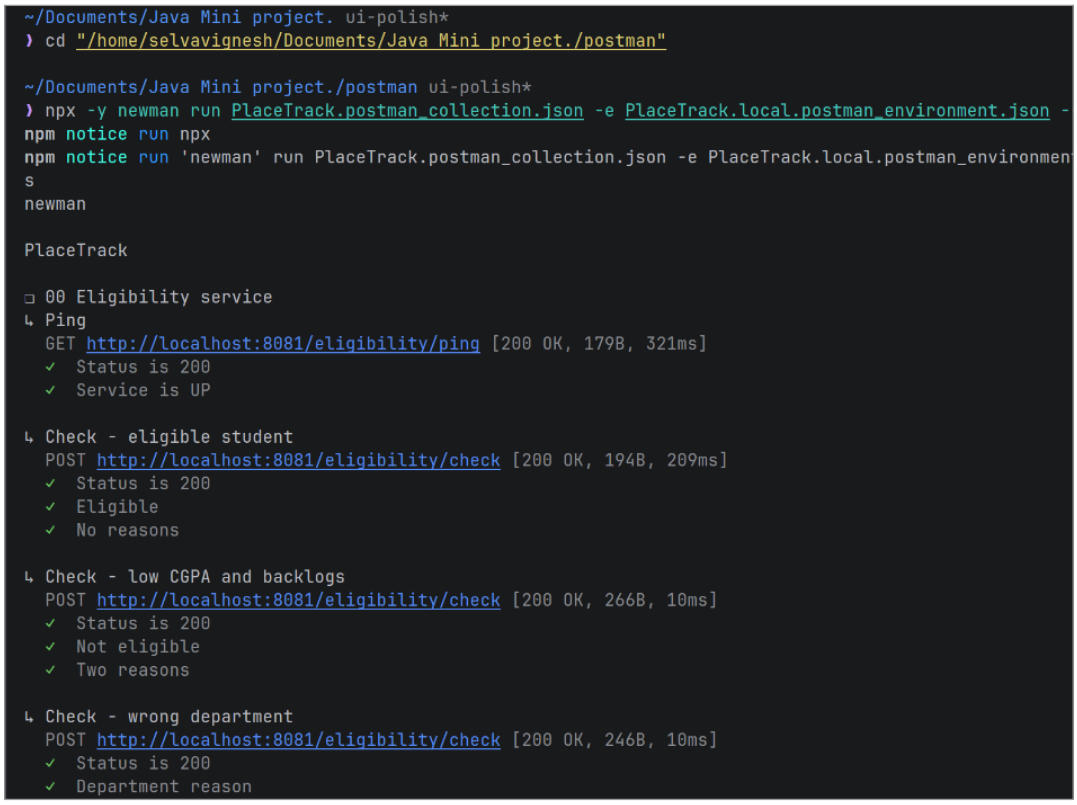


Figure 24: Newman run, folder 00: eligibility service requests and their test results

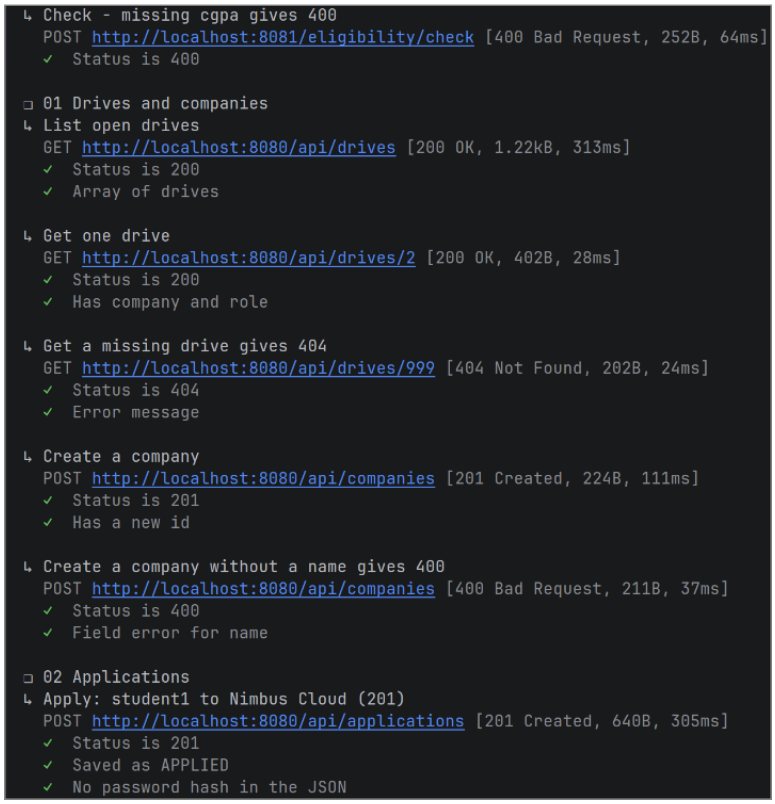


Figure 25: Newman run, folder 01: drives and companies (200, 404, 201, 400)

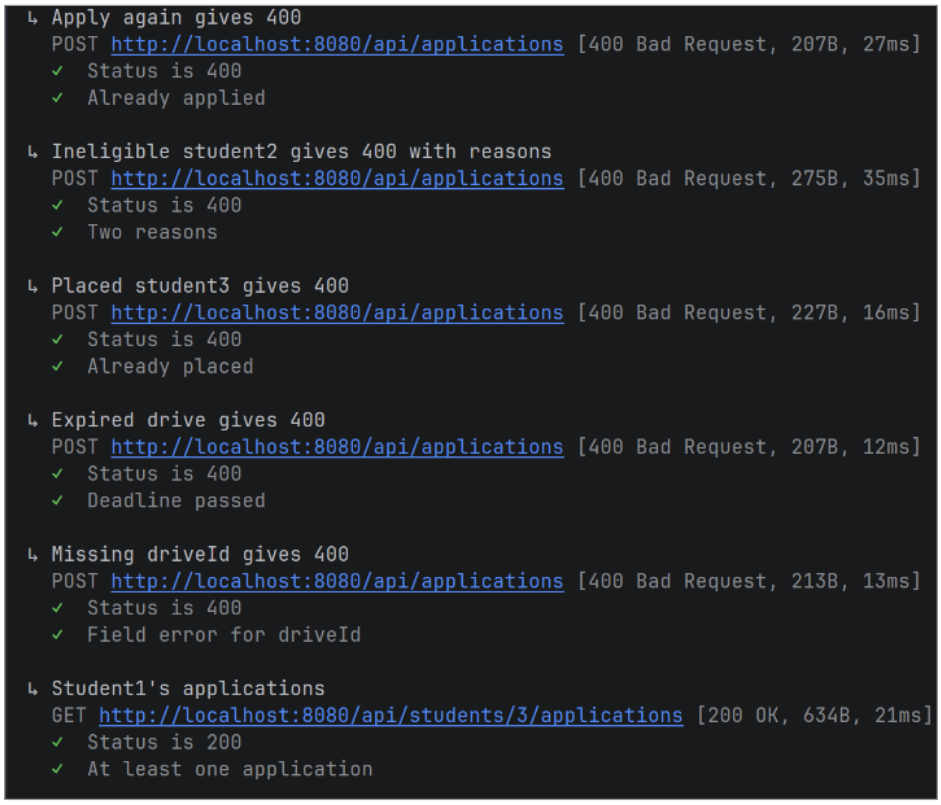
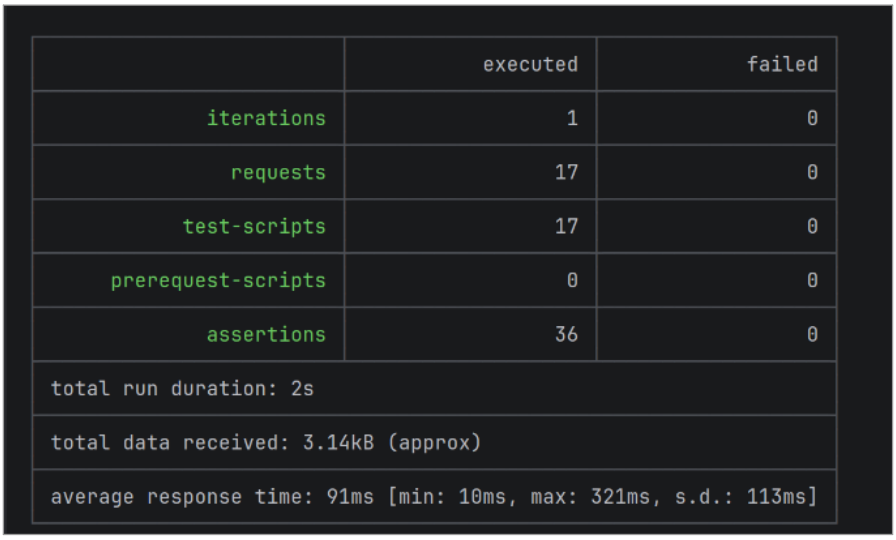


Figure 26: Newman run, folder 02: refused applications return 400 with the reason



	executed	failed
iterations	1	0
requests	17	0
test-scripts	17	0
prerequest-scripts	0	0
assertions	36	0
total run duration: 2s		
total data received: 3.14kB (approx)		
average response time: 91ms [min: 10ms, max: 321ms, s.d.: 113ms]		

Figure 27: Newman summary: 17 requests and 36 assertions, 0 failed

```
~/Documents/Java Mini project./postman ui-polish*
❯ npx -y newman run PlaceTrack.postman_collection.json -e PlaceTrack.local.postman_environment.json -
npm notice run npx
npm notice run 'newman' run PlaceTrack.postman_collection.json -e PlaceTrack.local.postman_environmen
newman

PlaceTrack

❏ 03 Service down (run by hand)
↳ Apply while the eligibility service is stopped gives 503
  POST http://localhost:8080/api/applications [503 Service Unavailable, 204B, 94ms]
  ✓  Status is 503
  ✓  Try-later message

┌─────────────────────────┬──────────┬──────────┐
│                         │ executed │   failed │
├─────────────────────────┼──────────┼──────────┤
│              iterations │        1 │        0 │
├─────────────────────────┼──────────┼──────────┤
│                requests │        1 │        0 │
├─────────────────────────┼──────────┼──────────┤
│            test-scripts │        1 │        0 │
├─────────────────────────┼──────────┼──────────┤
│      prerequest-scripts │        0 │        0 │
├─────────────────────────┼──────────┼──────────┤
│              assertions │        2 │        0 │
├─────────────────────────┴──────────┴──────────┤
│ total run duration: 164ms                     │
├───────────────────────────────────────────────┤
│ total data received: 52B (approx)             │
├───────────────────────────────────────────────┤
│ average response time: 94ms [min: 94ms, max: 94ms, s.d.: 0µs] │
└───────────────────────────────────────────────┘
```

Figure 28: Service-down test: POST /api/applications returns 503 while the eligibility service is stopped

13.5 Automated tests

```
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 4.840 s -- in com.pla
cetrack.eligibility.EligibilityControllerTest
[INFO] Running com.placetrack.eligibility.EligibilityServiceTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.036 s -- in com.pla
cetrack.eligibility.EligibilityServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jar:3.4.2:jar (default-jar) @ eligibility-service ---
[INFO] Building jar: /home/selvavignesh/Documents/Java Mini project./eligibility-service/tar
get/eligibility-service-0.0.1-SNAPSHOT.jar
[INFO]
[INFO] --- spring-boot:3.5.16:repackage (repackage) @ eligibility-service ---
[INFO] Replacing main artifact /home/selvavignesh/Documents/Java Mini project./eligibility-s
ervice/target/eligibility-service-0.0.1-SNAPSHOT.jar with repackaged archive, adding nested
dependencies in BOOT-INF/.
[INFO] The original artifact has been renamed to /home/selvavignesh/Documents/Java Mini proj
ect./eligibility-service/target/eligibility-service-0.0.1-SNAPSHOT.jar.original
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  12.068 s
[INFO] Finished at: 2026-10-18T18:38:48+05:30
[INFO] ------------------------------------------------------------------------
```

Figure 29: eligibility-service: 7 tests passed, BUILD SUCCESS

```
[INFO] Tests run: 24, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jar:3.4.2:jar (default-jar) @ placetrack-app ---
[INFO] Building jar: /home/selvavignesh/Documents/Java Mini project./placetrack-app/target/placetrack-
[INFO]
[INFO] --- spring-boot:3.5.16:repackage (repackage) @ placetrack-app ---
[INFO] Replacing main artifact /home/selvavignesh/Documents/Java Mini project./placetrack-app/target/p
NF/.
[INFO] The original artifact has been renamed to /home/selvavignesh/Documents/Java Mini project./place
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  26.387 s
[INFO] Finished at: 2026-10-19T11:34:20+05:30
[INFO] ------------------------------------------------------------------------
```

Figure 30: placetrack-app: 24 tests passed, BUILD SUCCESS

13.6 Database (H2 console)

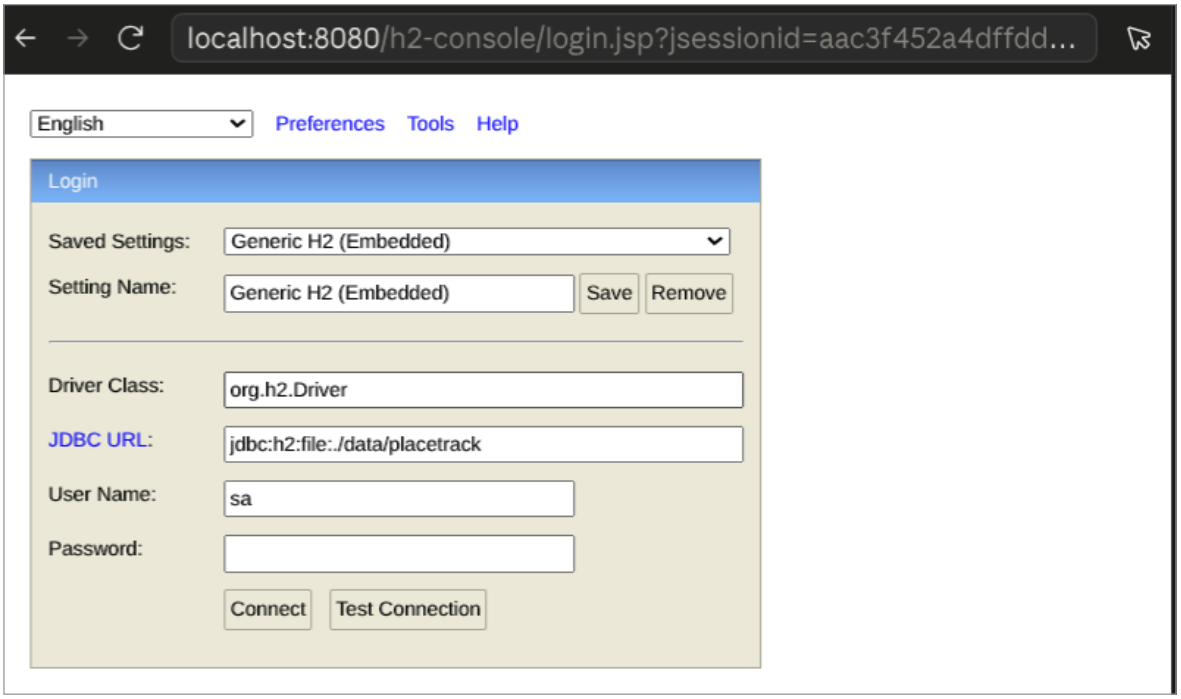


Figure 31: H2 console login with the JDBC URL of the project database

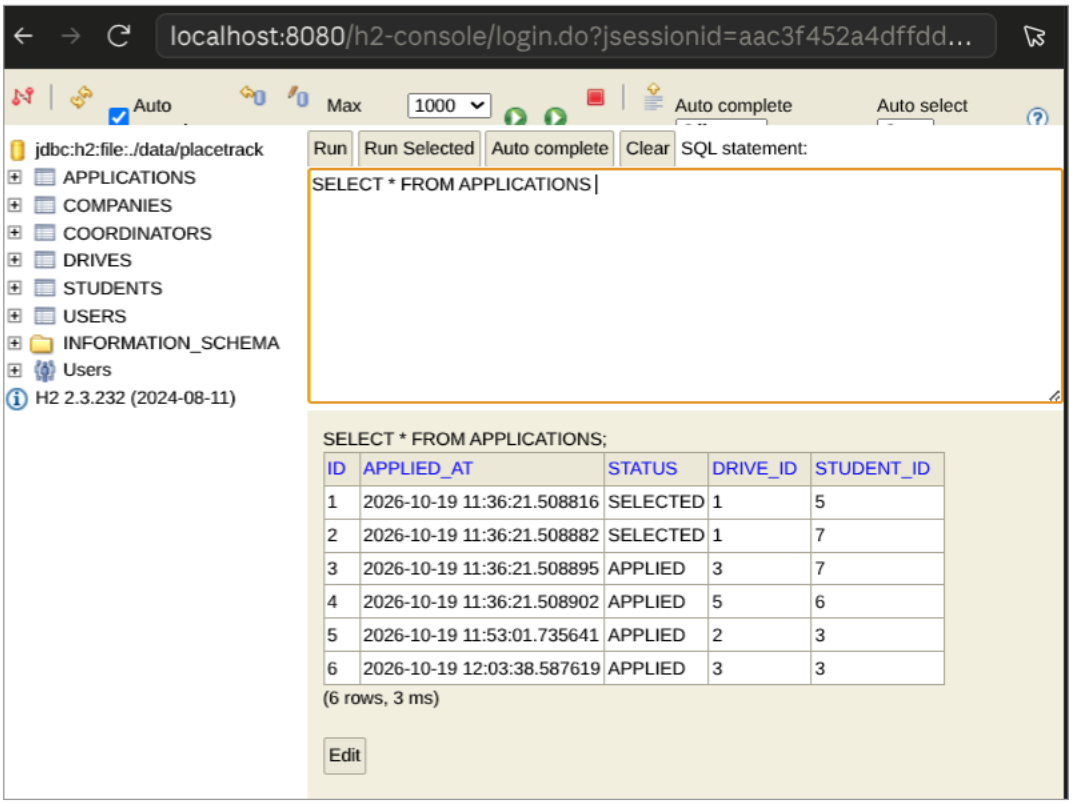


Figure 32: The six tables and the contents of APPLICATIONS

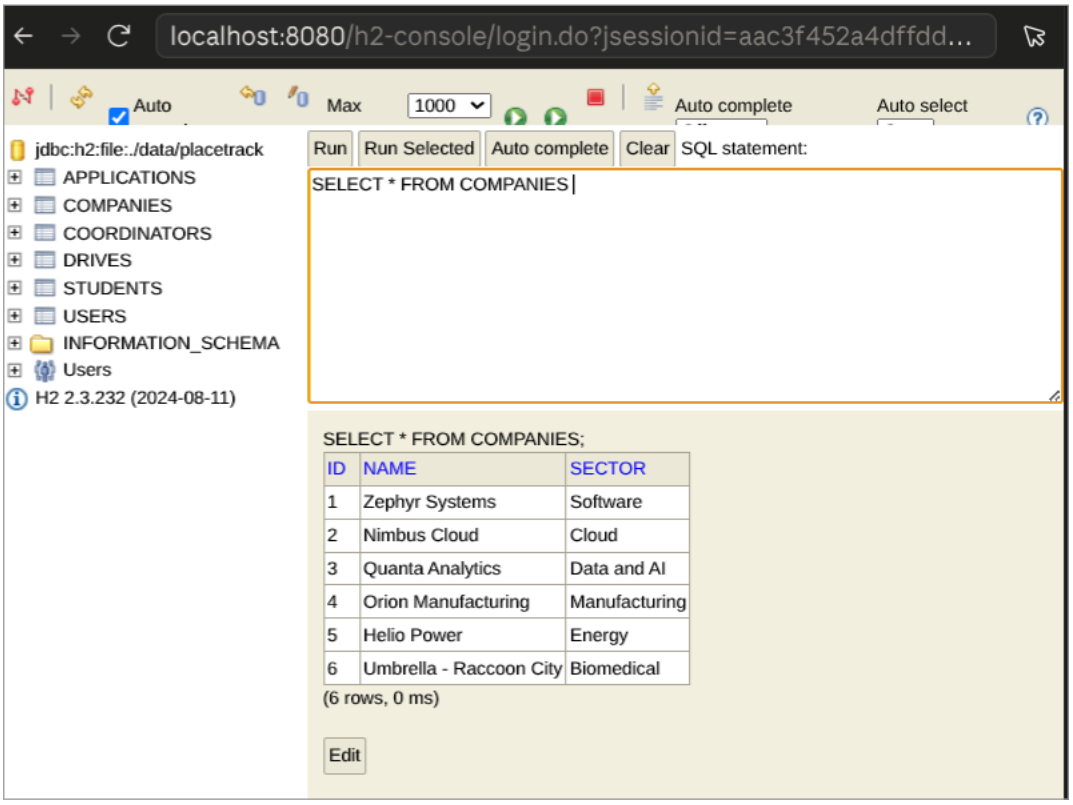


Figure 33: Contents of COMPANIES, including the company added during testing

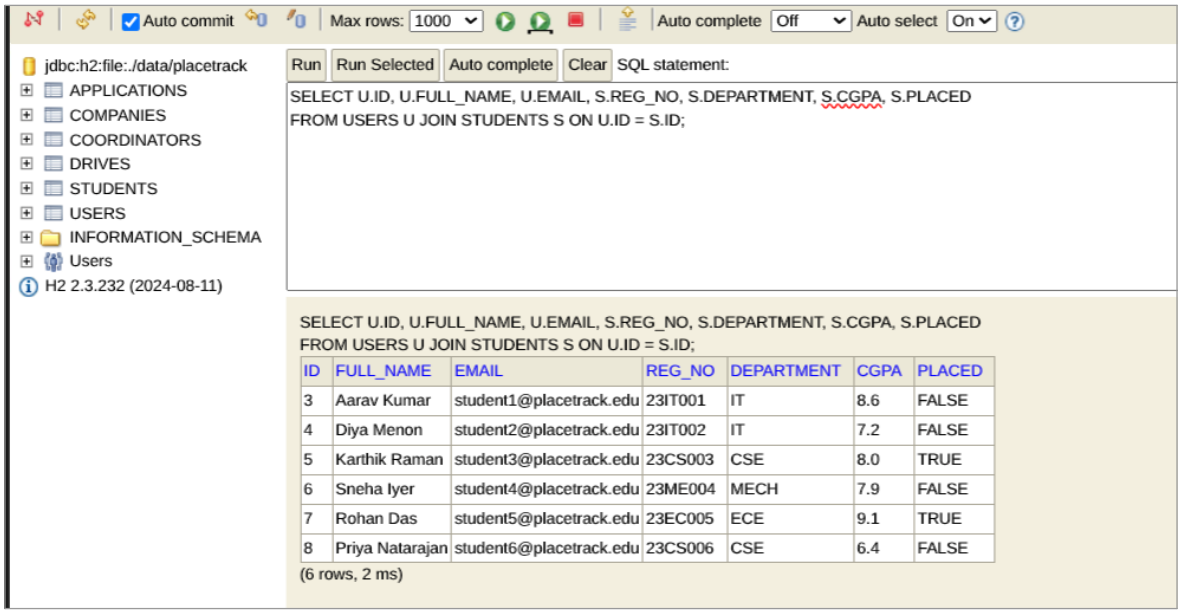


Figure 34: JOINED inheritance: each student is a row in USERS joined with a row in STUDENTS by the same id; PLACED is TRUE after selection

14. Individual Contributions

The work was divided into three modules so that each member owns one part of the application and one part of the syllabus.

Member	Name	Contribution
M1	Selvavignesh S	Data layer, inheritance, HQL, Session demo, eligibility microservice, company and drive pages
M2	Prakash B	Login and roles, apply flow, eligibility client, @Transactional, AOP, DI demo
M3	Krishna K	Applicants and status changes, profile, dashboard, REST API, Postman, README, report

Member	Concepts explained in the viva
M1	Entities, ORM mapping, ID strategy, CRUD, JOINED inheritance, HQL, SessionFactory, Session and Transaction, the eligibility microservice, monolith and microservices
M2	MVC request flow, login and roles, dependency injection, BeanFactory and ApplicationContext, AOP, @Transactional, the service call and its failure handling
M3	REST APIs, GET and POST, Postman and Newman, web services, REST and SOAP, Spring Cloud, Git, GitHub, Maven and DevOps

15. Conclusion

PlaceTrack meets its objectives. Coordinators manage companies, drives and applicants, and students apply only when they meet the criteria, with clear reasons when they do not. The project demonstrates every concept required for Units IV and V: a Spring Boot MVC application with JPA and Hibernate, ORM mapping with relationships, an ID strategy, inheritance, HQL, SessionFactory, Session and Transaction, dependency injection, AOP, REST APIs tested with Postman, a second service called over REST, and Git, GitHub and Maven.

Possible improvements are proper authentication for the REST API (Spring Security with JWT), a circuit breaker and service discovery with Spring Cloud, coordinator verification of student marks, e-mail notifications, and a continuous integration pipeline that runs the Maven and Newman tests on every push.

References

- Spring Boot Reference Documentation, https://docs.spring.io/spring-boot/
- Spring Framework Reference (Core, Web MVC, AOP), https://docs.spring.io/spring-framework/reference/
- Spring Data JPA Reference, https://docs.spring.io/spring-data/jpa/reference/
- Hibernate ORM User Guide, https://hibernate.org/orm/documentation/
- Thymeleaf documentation, https://www.thymeleaf.org/documentation.html
- H2 Database Engine, https://www.h2database.com/
- Postman Learning Center and Newman, https://learning.postman.com/